

AIU GYM

Multi-Department Gym Management System — Technical Documentation

A Laravel 13 application that manages a gym organization split into five independent departments: Trainers, Members, Branches, Workouts, and Warehouses. Each department has its own users, its own routes, and its own data model, but several models are shared or linked across departments (e.g. a Workout is visible to Trainers, Branches, and Members).

1. Tech Stack

Layer	Technology
Framework	Laravel 13 (PHP 8.3)
Auth	Laravel session auth (Auth::attempt), no Breeze/Jetstream scaffolding
Frontend	Blade + Bootstrap 5 (+ Bootstrap Icons), jQuery for AJAX filtering
Build	Vite + laravel-vite-plugin, Tailwind v4 (used only for a few Warehouse/Branch legacy views)
DB	Any Laravel-supported RDBMS (SQLite by default in tests)
Authorization	Laravel Gates/Policies (TrainerPolicy, SportsTypePolicy)
File storage	public disk (storage:link) for trainer images, member photos, workout images, warehouse brochures

2. Access Control Model

Every department is gated by two middlewares stacked in routes/web.php:

```
Route::middleware(['auth', 'department:<name>'])->group(function () {  
    // department routes  
});
```

auth — standard authenticated-session check.

department:<name> (App\Http\Middleware\DepartmentMiddleware) — compares the first URL segment against Auth::user()->department->department. A user whose department does not match the route prefix gets a 403.

Each User belongs to a role (employee | manager) and a department (trainers | members | branches | warehouses | workouts). Role determines what a user can do inside their department; department determines which department's routes they can even reach.

Seeded accounts (UserSeeder / Users.md): one manager + one employee per department, e.g. trainers.manager@gym.com / trainers.employee@gym.com, password "password" for all.

Login (LoginController::login) redirects post-auth based on the user's department:

```
'trainers' => redirect()->route('trainers.index'),  
'members' => redirect()->route('members.index'),
```

```
'branches' => redirect()->route('branches.dashboard'),
'warehouses' => redirect()->route('warehouse.dashboard'),
'workouts' => redirect()->route('workouts.dashboard'),
```

3. Core / Shared Tables

These aren't owned by a single module but are referenced by several:

Table	Purpose
users	App accounts. FKs: role_id, department_id.
roles	employee manager.
departments	trainers members branches warehouses workouts.
sports_types	Shared taxonomy used by Trainers (specialization) and Workouts (category).
countries	Shared by Branches and Warehouses.

App\Models\role and App\Models\department correspond to the roles and departments tables respectively.

4. Module: Trainers

4.1 Models

Model	File	Notes
Trainer	app/Models/Trainer.php	SoftDeletes, HasFactory
TrainerStatus	app/Models/TrainerStatus.php	Active / On Leave / Loaned / Ended Contract
SportsType	app/Models/SportsType.php	shared with Workouts module
TrainerWorkout	app/Models/TrainerWorkout.php	pivot model for Trainer <-> Workout

4.2 Database Table: trainers

Column	Type	Constraints
id	bigint	PK
firstname, lastname	string	required
fathername	string	nullable
phone	string	required
address	string	nullable
image	string	nullable, path on public disk
gender	string	required
sports_type_id	FK -> sports_types.id	indexed, onDelete('restrict')
birthplace	string	nullable

Column	Type	Constraints
birthdate	date	required
years_of_experience	integer	indexed
SSN	string	unique
email	string	unique, nullable
hiring_date	date	required
email_verified_at	timestamp	nullable
certification	enum(level_1..level_4)	default level_1
trainer_status_id	FK -> trainer_statuses.id	indexed, onDelete('restrict')
remember_token / timestamps / deleted_at	-	soft-deletable

4.3 Database Table: trainer_statuses

Column	Type	Constraints
id	bigint	PK
status	string	required
timestamps	-	-

4.4 Database Table: sports_types

Column	Type	Constraints
id	bigint	PK
type	string	required
timestamps	-	-

4.5 Database Table: trainer_workouts (pivot)

Column	Type	Constraints
id	bigint	PK
trainer_id	FK -> trainers.id	cascade
workout_id	FK -> workouts.id	cascade
timestamps	-	-

4.6 Relationships

Trainer	belongsTo	SportsType	(sports_type_id)
Trainer	belongsTo	TrainerStatus	(trainer_status_id)
Trainer	belongsToMany	Workout	(via trainer_workouts)
SportsType	hasMany	Trainer	
TrainerStatus	hasMany	Trainer	

Cross-module: Trainer <-> Workout (Workouts module) via `trainer_workouts`, and Trainer shares the `SportsType` taxonomy with Workout.

4.7 Controllers

TrainerController (app/Http/Controllers/TrainerController.php)

Method	Responsibility
index	Filtered/paginated listing (<code>Trainer::filter()</code> scope: specialty, min experience, status, name search). Returns partial HTML for AJAX requests (<code>trainers._trainer_grid</code>). Also computes <code>totalTrainers</code> / <code>activeTrainers</code> stat cards.
create / store	Create form + persist, with image upload to public disk.
show	Full profile incl. eager-loaded <code>workouts.sportsType</code> , <code>workouts.workoutLevel</code> .
edit / update	<code>Gate::authorize('edit', \$trainer)</code> on <code>edit()</code> . <code>update()</code> conditionally includes <code>trainer_status_id</code> in the validated fields only if <code>Gate::allows('editStatus', \$trainer)</code> , so status changes stay manager-only.
destroy	<code>Gate::authorize('delete', \$trainer)</code> ; blocks deletion if <code>\$trainer->workouts()->exists()</code> , redirecting to <code>trainers.show</code> with an error flash.
updateStatus	Dedicated quick-status endpoint used by the inline <code><select></code> on the show page; <code>Gate::authorize('editStatus', \$trainer)</code> .
specialties / createSpecialty / editSpecialties / deleteSpecialties	CRUD over <code>SportsType</code> , gated by <code>SportsTypePolicy</code> (manager-only).

4.8 Policy / Authorization

`app/Policies/TrainerPolicy.php` (`SportsTypePolicy` mirrors this: `viewAny` / `create` / `update` / `delete` all manager-only):

Ability	Rule
view, edit	any authenticated employee or manager
delete, editStatus	manager only
editSpecialization (on <code>SportsType</code>)	manager only

4.9 Routes

Prefix requires `department:trainers`

Method	URI	Name	Action
PATCH	<code>/trainers/{trainer}/status</code>	<code>trainers.updateStatus</code>	<code>TrainerController@updateStatus</code>
GET	<code>/trainers/specialties</code>	<code>trainers.specialties</code>	<code>TrainerController@specialties</code>

Method	URI	Name	Action
POST	/trainers/specialties	trainers.createSpecialty	TrainerController@createSpecialty
PUT	/trainers/specialties/{specialty}	trainers.editSpecialties	TrainerController@editSpecialties
DELETE	/trainers/specialties/{specialty}	trainers.deleteSpecialties	TrainerController@deleteSpecialties
RESOURCE	/trainers[...]	trainers.*	Route::resource('trainers', TrainerController::class)

5. Module: Members

5.1 Models

Model	File	Notes
Member	app/Models/Member.php	-
MembershipType	app/Models/MembershipType.php	-
MemberStatus	app/Models/MemberStatus.php	-

5.2 Database Table: members

Column	Type	Constraints
id	bigint	PK
first_name, father_name, last_name	string	required
email	string	required (not unique-constrained at DB level)
birth_date	date	required
national_id	string(11)	unique
phone	string	required
photo	string	nullable, path on public disk
membership_duration	integer	months
membership_type_id	FK -> membership_types.id	required
member_status_id	FK -> member_statuses.id	required
timestamps	-	-

5.3 Database Table: membership_types

Column	Type	Constraints
id	bigint	PK
name	string	required
timestamps	-	-

5.4 Database Table: member_statuses

Column	Type	Constraints
id	bigint	PK
name	string	required
timestamps	-	-

5.5 Database Table: member_workout (pivot)

Column	Type	Constraints
id	bigint	PK
member_id	FK -> members.id	cascade
workout_id	FK -> workouts.id	cascade
trainer_id	FK -> trainers.id	nullable, nullOnDelete
start_date	date	required
timestamps	-	-
unique constraint	-	unique(member_id, workout_id)

5.6 Relationships

Member belongsTo MembershipType
Member belongsTo MemberStatus
Member belongsToMany Workout (via member_workout, pivot: trainer_id, start_date)

Cross-module: the member_workout pivot links Members directly to Workouts, and optionally records which trainer (from the Trainers module) is coaching that member/workout pairing — the only table that touches three departments' data at once (Members, Workouts, Trainers).

5.7 Controllers

MemberController (app/Http/Controllers/MemberController.php)

Method	Responsibility
index	Filters by first_name, last_name, national_id, member_status_id. Not paginated (uses ->get()).
create / store	Validates national_id (11 digits, unique), phone (09\d{8}), age 18–70 via birth_date bounds; uploads photo; hardcodes new

Method	Responsibility
	members to member_status_id = 1.
edit / update	Unique checks ignore current record id (unique:members,email,{id} style).
destroy	Deletes photo from public disk before deleting the record.
show	Eager-loads membershipType; computes expiry as created_at->addMonths(membership_duration) inline in the Blade view.

5.8 Routes

Prefix requires department:members

Method	URI	Name	Action
GET	/members/create	—	MemberController@create
POST	/members	—	MemberController@store
GET	/members	members.index	MemberController@index
DELETE	/members/{id}	—	MemberController@destroy
GET	/members/{id}/edit	—	MemberController@edit
PUT	/members/{id}	—	MemberController@update
GET	/members/{id}	—	MemberController@show

6. Module: Branches

6.1 Models

Model	File	Notes
Branch	app/Models/Branch.php	-
Country	app/Models/Country.php	shared with Warehouses

6.2 Database Table: branches

Column	Type	Constraints
id	bigint	PK
name	string	required
location	string	nullable
phone	string	nullable
governorate	string	nullable
country_id	FK -> countries.id	required
capacity	integer	required
brochure_path	string	nullable

Column	Type	Constraints
timestamps	-	-

6.3 Database Table: countries

Column	Type	Constraints
id	bigint	PK
name	string	required
timestamps	-	-

6.4 Database Table: branch_warehouse (pivot)

Column	Type	Constraints
id	bigint	PK
branch_id	FK -> branches.id	cascade
warehouse_id	FK -> warehouses.id	cascade
timestamps	-	-

6.5 Database Table: branch_workout (pivot)

Column	Type	Constraints
id	bigint	PK
branch_id	FK -> branches.id	cascade
workout_id	FK -> workouts.id	cascade
timestamps / unique	-	unique(branch_id, workout_id)

6.6 Relationships

Branch belongsTo Country
 Branch belongsToMany Warehouse (via branch_warehouse)
 Branch belongsToMany Workout (via branch_workout)

Cross-module: Branches connect to Warehouses (many-to-many, a branch can be supplied by several warehouses and vice versa) and to Workouts (which sessions are offered at which branch).

6.7 Controllers

BranchController (app/Http/Controllers/BranchController.php)

Method	Responsibility
index	Search by name/location, paginated (12/page), eager-loads country.
create / store	Uploads brochure to public disk under brochures/.

Method	Responsibility
show, edit, update	Standard CRUD; update re-uploads brochure to public disk if a new file is present.
destroy	Deletes the branch record.

6.8 Routes

Prefix requires department:branches. Note: legacy/manual dashboard views (branches.dashboard, data.entry, edit.data, details) are static/simplified Blade pages not backed by BranchController — they're closures defined directly in routes/web.php.

Method	URI	Name	Action
GET	/branches/dashboard	branches.dashboard	closure -> branches.dashboard view
GET	/branches/data-entry	data.entry	redirects to branches.create
GET	/branches/edit-data	edit.data	closure -> branches.edit-data view
GET	/branches/details	details	closure -> branches.details view
RESOURCE	/branches[...]	branches.*	Route::resource('branches', BranchController::class)

7. Module: Workouts

7.1 Models

Model	File	Notes
Workout	app/Models/Workout.php	-
WorkoutLevel	app/Models/WorkoutLevel.php	-
SportsType	app/Models/SportsType.php	shared with Trainers

7.2 Database Table: workouts

Column	Type	Constraints
id	bigint	PK
name	string(255)	unique
description	text	nullable
price	decimal(10,2)	required
duration	integer	required (minutes)
sportstype_id	FK -> sports_types.id	onDelete('cascade')
workoutlevel_id	FK ->	onDelete('cascade')

Column	Type	Constraints
	workout_levels.id	
image	string	nullable
start_date	datetime	required
timestamps / deleted_at	-	soft-deletable

7.3 Database Table: workout_levels

Column	Type	Constraints
id	bigint	PK
level	enum(beginner,intermediate,advanced)	default beginner
timestamps	-	-

7.4 Relationships

Workout	belongsTo	SportsType	(sportstype_id)
Workout	belongsTo	WorkoutLevel	(workoutlevel_id)
Workout	belongsToMany	Trainer	(via trainer_workouts)
Workout	belongsToMany	Branch	(via branch_workout)
Workout	belongsToMany	Member	(via member_workout, pivot: trainer_id, start_date)

Pivots owned by other modules but attached to Workout: trainer_workouts, branch_workout, member_workout (see respective module sections).

Workout is the most connected model in the system — it is the join point between Trainers, Branches, and Members.

7.5 Controllers

WorkoutController (app/Http/Controllers/WorkoutController.php)

Method	Responsibility
index	Paginated (9/page) listing, eager-loads sportsType, workoutLevel.
create / store	Uses StoreWorkoutRequest (Form Request); uploads image; syncs branches[] on create.
show	Eager-loads sportsType, workoutLevel, branches, trainers, members.
edit / update	Uses UpdateWorkoutRequest; deletes old image before storing a new one; re-syncs branches.
destroy	Blocks deletion if branches()->exists() or trainers()->exists(); deletes image from disk otherwise.
searchPage	Renders the advanced-search UI.
search	AJAX endpoint (Workout::filter() scope: sports_type_id, workout_level_id, max_price); returns rendered partial HTML +

Method	Responsibility
	JSON envelope.

WorkoutDashboardController (app/Http/Controllers/WorkoutDashboardController.php)

Computes \$stats (workouts_count, sports_types_count, workout_levels_count, trainers_count) for the workouts.dashboard Blade view.

7.6 Routes

Prefix requires department:workouts

Method	URI	Name	Action
GET	/workouts/dashboard	workouts.dashboard	WorkoutController@index
GET	/workouts/search	workouts.search.page	WorkoutController@searchPage
GET	/workouts/search/ajax	workouts.search	WorkoutController@search
RESOURCE	/workouts[...]	workouts.*	Route::resource('workouts', WorkoutController::class)

8. Module: Warehouses

8.1 Models

Model	File	Notes
Warehouse	app/Models/Warehouse.php	-
Country	app/Models/Country.php	shared with Branches

8.2 Database Table: warehouses

Column	Type	Constraints
id	bigint	PK
name	string	required
location	string	required
phone	string	required
country_id	FK -> countries.id	required
governorate	string	nullable
capacity	integer	nullable
brochure	string	nullable
timestamps	-	-

8.3 Relationships

Warehouse belongsTo Country
Warehouse belongsToMany Branch (via branch_warehouse)

Shares the branch_warehouse pivot with the Branches module.

8.4 Controllers

WarehouseController (app/Http/Controllers/WarehouseController.php)

Method	Responsibility
index	Filters: search (name), governorate, capacity (min), country_id; paginated (10/page); also computes summary stats and a hardcoded Syrian-governorate list for the filter dropdown.
create / store	Validates name unique, governorate against a fixed in: list, brochure (pdf/jpg/jpeg/png <=5MB); syncs branches[].
edit / update	Same validation with Rule::unique(...)->ignore(\$warehouse->id); deletes the old brochure from the public disk before storing the new one; sync()s branches or detach()es if none submitted.
downloadBrochure	Streams the brochure file via response()->download().
destroy	Deletes the brochure file, then the record.

DashboardController (app/Http/Controllers/DashboardController.php)

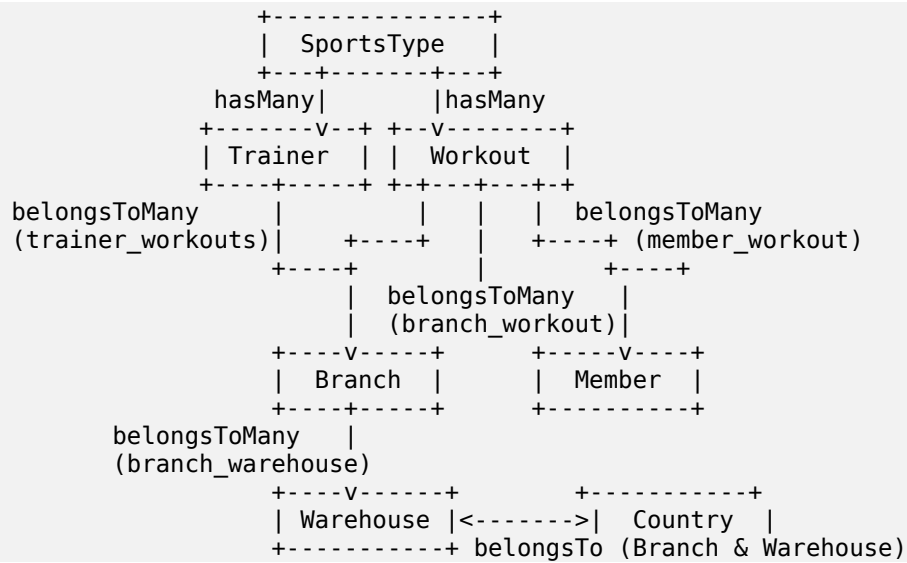
Powers the warehouse.dashboard route: totalWarehouses, totalCountries, totalBrochures, latestWarehouses (5 most recent, with country).

8.5 Routes

Prefix requires department:warehouses

Method	URI	Name	Action
GET	/warehouses	warehouses.index	WarehouseController@index
GET	/warehouses/dashboard	warehouse.dashboard	DashboardController@index
GET	/warehouses/create	warehouses.create	WarehouseController@create
POST	/warehouses	warehouses.store	WarehouseController@store
GET	/warehouses/{warehouse}/edit	warehouses.edit	WarehouseController@edit
PUT	/warehouses/{warehouse}	warehouses.update	WarehouseController@update
DELETE	/warehouses/{warehouse}	warehouses.destroy	WarehouseController@destroy
GET	/warehouses/{warehouse}/download	warehouses.download	WarehouseController@downloadBrochure

9. Cross-Module Relationship Map



10. Setup & Installation

```
composer install
cp .env.example .env
php artisan key:generate
# configure DB credentials in .env
php artisan migrate --seed
php artisan storage:link
npm install
npm run build # or `npm run dev` for local development
```

On Windows with Laravel Herd, run the following if uploaded images are not rendering after seeding:

```
rmdir public\storage
php artisan storage:link
```

Seeded credentials (database/seeder/UserSeeder.php): {department}.manager@gym.com / {department}.employee@gym.com, password "password", for each of the five departments.